

toy-model

September 11, 2025

```
[ ]: def calculate_metabolome_from_net(f, ct_hyp, diet, net, in_degree_flag,
    ↪ MAX_ID_metabolites, MAX_ID_celltypes):
    """
    This is a function used to create sparse matrices made of metabolites and
    ↪ celltypes
    where metabolite consumption and production is considered. The matrices
    ↪ created are "m2b" and "b2m":
    (1) m2b is a matrix determines nutrient uptake by celltypes, and
    (2) b2m is a matrix determines the nutrient secretion.
    Both matrices have rows representing cancer celltypes and columns
    ↪ representing metabolites.
    Two matrices are created based on (1) the metabolite consumption and
    ↪ production network which is
    encode in "net" as a dataframe, and (2) the hypothesised relative cell type
    ↪ frequencies "ct_hyp". Those metabolites that are not taken up by any
    ↪ celltype are inherited as is from the diet supplied, while uptake of
    ↪ metabolites is assumed to be complete i.e., even if only a single celltype
    ↪ can take up a metabolite, it consumes all of it from the environment.
    """
    ##### A^in
    valid_index = np.where((net['edgeType']==2) | (net['edgeType']==5))[0]
    col = net['metabolites'].iloc[valid_index]
    row = net['celltypes'].iloc[valid_index]
    data = np.ones((len(valid_index),))
    m2b = csr_matrix((data,(row,col)), shape=(MAX_ID_celltypes,
    ↪ MAX_ID_metabolites)).toarray()#.todense()
    in_degree = m2b.sum(0)

    ##### A^out
    valid_index = np.where((net['edgeType']==3) | (net['edgeType']==5))[0]
    col = net['metabolites'].iloc[valid_index]
    row = net['celltypes'].iloc[valid_index]
    data = np.ones((len(valid_index),))
    b2m = csr_matrix((data,(row,col)), shape=(MAX_ID_celltypes,
    ↪ MAX_ID_metabolites)).toarray()#.todense()

    if in_degree_flag:
```

```

        m2b = m2b/in_degree

        ##### Intake matrix calculation
        cellnum = np.matlib.repmat(ct_hyp[:, np.newaxis], 1, MAX_ID_metabolites)
        con_matrix = m2b*cellnum
        ### Normalising consumption of metabolites by the total number of consumers
        ↪ of each metabolite
        total_cellnum = con_matrix.sum(0)
        tau = np.zeros(len(total_cellnum))
        i_nonzero = np.where(total_cellnum > 0)[0]
        tau[i_nonzero] += 1/total_cellnum[i_nonzero]
        con_norm = con_matrix * tau
        intake_vector = np.dot(con_norm, diet.values)

        ##### Output matrix
        out_degree = b2m.sum(1)
        inv_out_degree = np.zeros(len(out_degree))
        i_nonzero = np.where(out_degree > 0)[0]
        inv_out_degree[i_nonzero] += f/out_degree[i_nonzero] # Secretion by each
        ↪ celltype is split equally between all the metabolites it secretes
        out_mult = np.matlib.repmat(inv_out_degree[:, np.newaxis], 1,
        ↪ MAX_ID_metabolites)
        output_matrix = b2m * out_mult

        ##### Final metabolome-secreted + unused
        secreted_metabolome = np.dot(output_matrix.T, intake_vector)
        i_unused = np.where(in_degree == 0)[0] # Metabolites not consumed by any
        ↪ celltype
        metabolome_unused = np.zeros(len(diet.values))
        metabolome_unused[i_unused] += diet.values[i_unused]

        metabolome_pred = secreted_metabolome + metabolome_unused

        return [m2b, b2m, metabolome_pred]

```

```
[ ]: f = 0.5
```

```
[ ]: ct_hyp = np.array([100., 200.])
```

```
[ ]: diet = pd.Series([10., 20., 25.])
```

```
[ ]: in_degree_flag = False
```

```
[ ]: net_test
```

```
[ ]:
      metabolites  celltypes  edgeType
0              0          0          0
```

1	1	0	2
2	2	0	2
3	0	1	0
4	1	1	0
5	2	1	2
6	0	0	3
7	1	0	3
8	2	0	0
9	0	1	3
10	1	1	0
11	2	1	0

```
[ ]: MAX_ID_celltypes
```

```
[ ]: 2
```

```
[ ]: MAX_ID_metabolites
```

```
[ ]: 3
```

```
[ ]: ##### A^in as inferred from net_test
calculate_metabolome_from_net(f, ct_hyp, diet, net_test, in_degree_flag,
↪MAX_ID_metabolites, MAX_ID_celltypes)[0]
```

```
[ ]: array([[0., 1., 1.],
         [0., 0., 1.]])
```

```
[ ]: ##### A^out as inferred from net_test
calculate_metabolome_from_net(f, ct_hyp, diet, net_test, in_degree_flag,
↪MAX_ID_metabolites, MAX_ID_celltypes)[1]
```

```
[ ]: array([[1., 1., 0.],
         [1., 0., 0.]])
```

```
[ ]: ##### Final predicted metabolome-secreted + unused
calculate_metabolome_from_net(f, ct_hyp, diet, net_test, in_degree_flag,
↪MAX_ID_metabolites, MAX_ID_celltypes)[2]
```

```
[ ]: array([25.41666667,  7.08333333,  0.          ])
```